

SIMATSENGINEERING

(Saveetha School of Engineering)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1

Git & Docker Technical Assignment

Technical Report

Project: Student Attendance & Result Management System

Submitted By	Y. Rishitha Reddy
Register No.	192424233
Department	Artificial Intelligence and Data Science
Subject	Software Engineering
Faculty	Dr. Latha R.
College	SIMATS Engineering
Academic Year	2024 – 2028
Course Code / CO Assessed	CSA10 / CO3
Weightage / Total Marks	20% / 25

Table of Contents

1. Problem Analysis 3

2. Project Structure Design 4

3. Git Repository Initialization 5

4. Git Branching Strategy 6

5. Version Control Workflow 7

6. Commit History Analysis 8

7. Docker Environment Design 9

8. Dockerfile Development 10

9. Docker Compose Design 11

10. Container Deployment and Testing 13

11. Benefits of Git and Docker 14

12. Challenges and Improvements 15

Expected Deliverables 16

Conclusion 16

1. Problem Analysis

1.1 Problem Statement

Educational institutions still rely heavily on manual or spreadsheet-based methods to record daily student attendance and to compute, store, and publish semester results. This approach is time-consuming, error-prone, difficult to audit, and does not scale well across multiple departments, sections, and academic years. The assigned project is to design and develop a Student Attendance & Result Management System (ARMS) — a full-stack web application that allows faculty to mark attendance digitally, allows administrators to manage courses and students, and allows results to be uploaded, computed, and published in a consistent and traceable manner.

1.2 Project Objectives

- Provide a centralized, web-based platform for recording and monitoring daily student attendance.
- Automate calculation of attendance percentage and flag students who fall below the minimum eligibility criteria.
- Enable faculty to upload subject-wise marks and automatically compute grades, GPA, and result status.
- Provide role-based access for Admin, Faculty, and Students with secure authentication.
- Containerize the application using Docker so that it can be deployed consistently across development, testing, and production environments.
- Apply Git version control with a structured branching strategy to support collaborative, traceable development.

1.3 Functional Requirements

- User registration and role-based login (Admin / Faculty / Student) secured using JWT authentication.
- Faculty can create/select a course session and mark attendance (Present/Absent/Late) for each enrolled student.
- System automatically calculates per-subject and overall attendance percentage for every student.
- Faculty can upload internal and external marks; the system computes grade and result status (Pass/Fail).
- Students can view their own attendance history and result report card.
- Admin can manage students, courses, and generate consolidated reports (CSV/PDF export).
- The system sends a notification/flag when a student's attendance drops below 75%.

1.4 Expected Outcomes

The final deliverable is a containerized, version-controlled full-stack application in which the frontend, backend API, and database each run as an independent Docker service orchestrated through Docker Compose. The Git repository will contain a complete, meaningful commit history following a Gitflow-inspired branching model (main, develop, feature, release, hotfix), demonstrating collaborative software engineering practice as required by CO3 — building full-stack applications with an emphasis on

containerization, version control, and collaborative development.

2. Project Structure Design

The repository is organized as a monorepo containing two independently deployable applications — a React.js client and a Node.js/Express server — along with shared configuration for Docker and CI/CD. Separating client and server into their own directories, each with its own Dockerfile, allows each service to be built, versioned, and scaled independently while still being managed from a single Git repository.

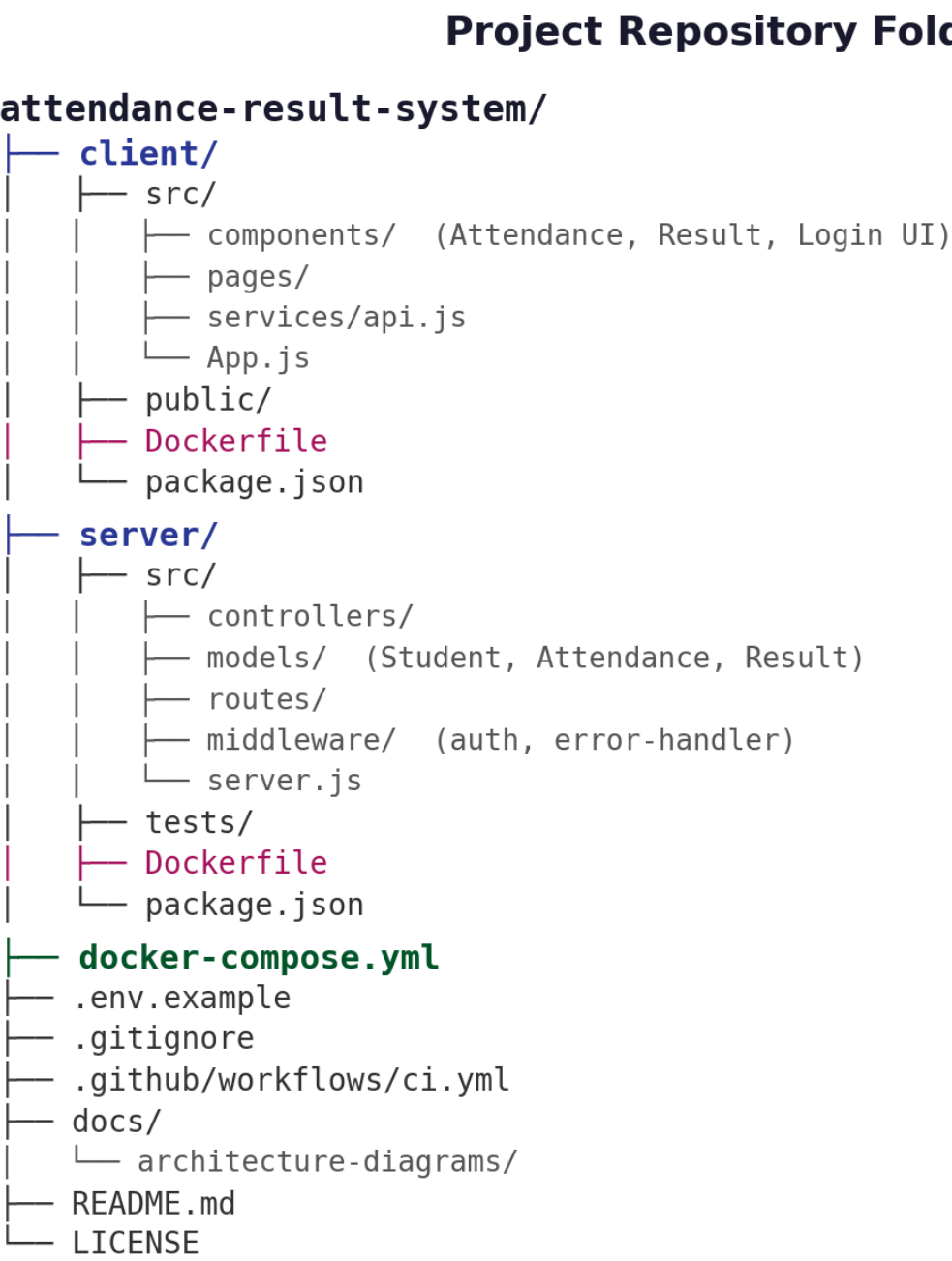


Figure 2.1: Project repository folder structure

2.1 Purpose of Major Folders and Files

Folder/File	Purpose
client/src/components/	Reusable React UI components (attendance grid, result card, login form, navbar).
client/src/pages/	Top-level route views such as Dashboard, Attendance, Results, and Admin pages.
client/src/services/api.js	Centralized Axios instance for calling the backend REST API.
server/src/controllers/	Request handlers implementing business logic for attendance and result operations.
server/src/models/	Mongoose schemas for Student, Course, Attendance, and Result collections.

server/src/routes/	Express route definitions mapping HTTP endpoints to controllers.
server/src/middleware/	JWT authentication guard, role-based access control, and centralized error handling.
server/tests/	Unit and integration tests for controllers and models.
client/Dockerfile & server/Dockerfile	Instructions to build production-ready container images for each service.
docker-compose.yml	Defines and wires together the client, server, database, and proxy containers.
.env.example	Template listing required environment variables without exposing real secrets.
.gitignore	Excludes node_modules, build output, and .env files from version control.
.github/workflows/ci.yml	Continuous-integration pipeline that lints, tests, and builds Docker images on every push.
docs/	Architecture diagrams and supplementary technical documentation.
README.md	Setup, run, and contribution instructions for the repository.

3. Git Repository Initialization

3.1 Repository Initialization Steps

The repository was initialized locally and then connected to a remote hosted on GitHub, following the sequence below:

```
$ mkdir attendance-result-system && cd attendance-result-system
$ git init
$ git branch -M main
$ echo "node_modules\n.env\ndist\nbuild/" > .gitignore
$ git add .gitignore README.md
$ git commit -m "chore: initial commit — project scaffolding"
$ git remote add origin https://github.com/rishitha-reddy/attendance-result-system.git
$ git push -u origin main
```

3.2 Purpose of Essential Git Files

File	Purpose
.git/	Hidden directory created by 'git init' that stores the entire object database, refs, and history of the repository.

.gitignore	Prevents environment files, dependency folders (node_modules), and build artifacts from being tracked, keeping the repository clean and secure.
README.md	Documents the project purpose, setup steps, and usage instructions for collaborators.
.git/config	Stores repository-level configuration such as the remote origin URL and branch tracking information.
HEAD	Points to the branch or commit currently checked out in the working directory.

Initializing Git at the very start of the project ensures that every subsequent change — from scaffolding to the final Docker configuration — is captured in a complete, auditable history, which is essential for collaborative development and for satisfying the CO3 requirement of version-controlled full-stack development.

4. Git Branching Strategy

A Gitflow-inspired branching model was adopted for this project because it cleanly separates ongoing development from stable, released code and gives the team a safe channel to patch production issues without disrupting in-progress feature work.

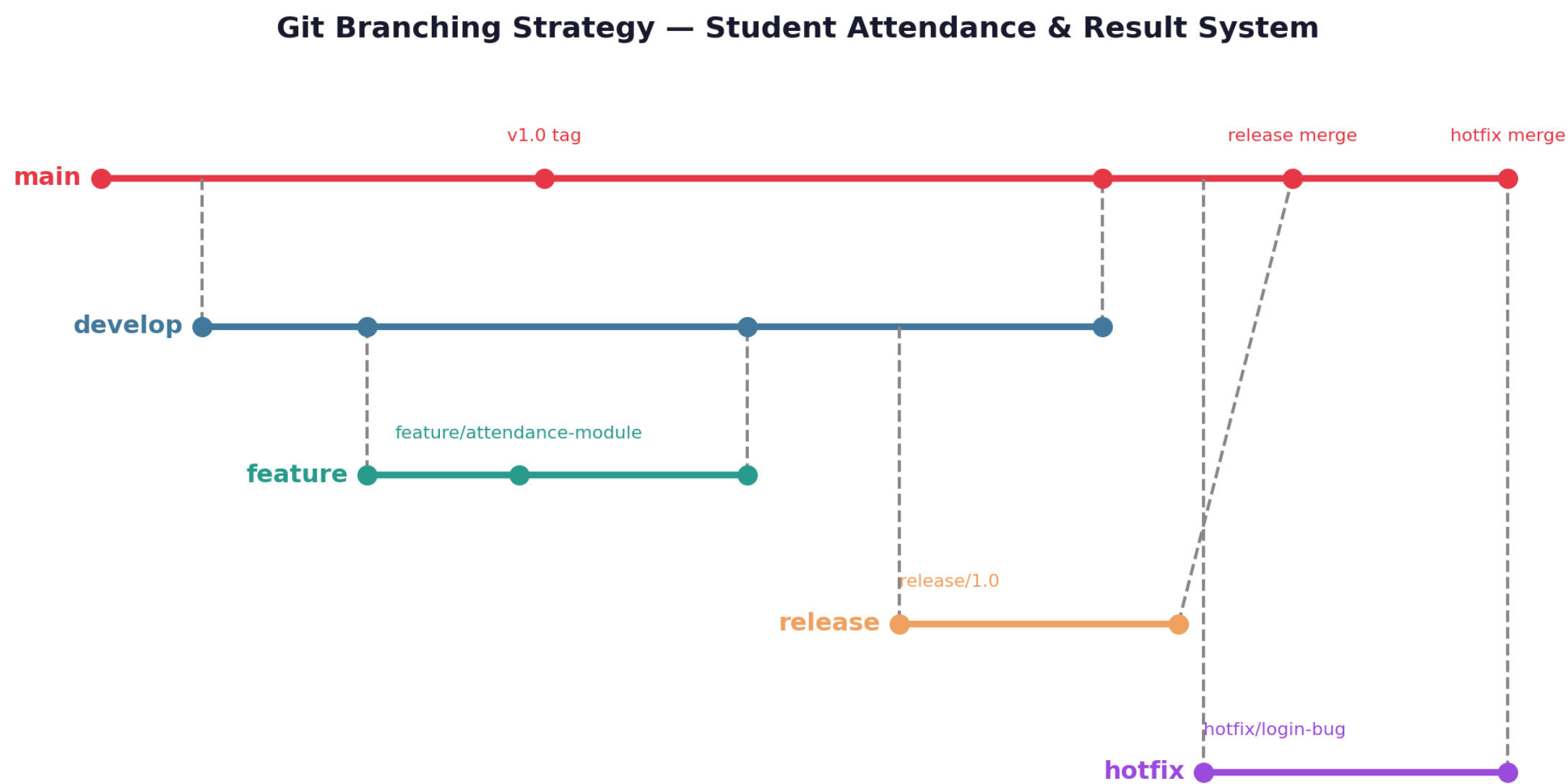


Figure 4.1: Git branching strategy for the Attendance & Result System

4.1 Branch Roles

Branch	Purpose
main	Always reflects the latest stable, production-ready release. Every commit on main is tagged with a version

	number.
develop	Integration branch where completed features are merged and tested together before a release is cut.
feature/*	Short-lived branches created from develop for a single feature (e.g., feature/attendance-module, feature/result-module). Merged back into develop via pull request.
release/*	Branched from develop when preparing a new version; used for final QA, version bumps, and bug fixes prior to release.
hotfix/*	Branched directly from main to urgently patch a production defect, then merged into both main and develop.

4.2 Justification

- The project has two independently developed modules (Attendance and Result) — feature branches let each be built and tested in isolation without blocking the other.
- develop acts as a safe integration point, so main is never broken and can be deployed via Docker at any time.
- release/* branches give a dedicated window to update the Docker image tags and docker-compose.yml for a version before it ships.
- hotfix/* supports rapid, isolated correction of production defects (e.g., an incorrect attendance percentage calculation) without pulling in unfinished develop work.
- This model maps naturally onto Docker image tagging: images built from main are tagged as stable releases (e.g., v1.0), while images from develop can be tagged 'latest-dev' for internal testing.

5. Version Control Workflow

The development workflow follows a consistent feature-branch cycle: create a branch from develop, commit incremental changes, open a pull request, resolve any conflicts, and merge. The key operations performed during this project are summarized below.

5.1 Representative Workflow Commands

```
# Start a new feature branch
$ git checkout -b feature/attendance-module develop
$ git pull origin develop
$ git checkout -b feature/attendance-module

# Work on the feature and commit incrementally
$ git add server/src/models/Student.js server/src/models/Attendance.js
$ git commit -m "feat: add student and course models"
$ git add server/src/controllers/attendanceController.js
$ git commit -m "feat: implement daily attendance marking API"
```

```
# Synchronize with the team
$ git push origin feature/attendance-module

# Merge back after review
$ git checkout develop
$ git merge --no-ff feature/attendance-module
$ git push origin develop
```

5.2 Conflict Resolution Example

A merge conflict occurred when both the attendance and result feature branches modified the shared Student.js model. Git flagged the conflicting hunk between the two model definitions; the conflict was resolved manually by retaining both the attendance and result reference fields, followed by staging and committing the resolution:

```
$ git merge feature/result-module
Auto-merging server/src/models/Student.js
CONFLICT (content): Merge conflict in server/src/models/Student.js
# ... conflict markers resolved manually in editor ...
$ git add server/src/models/Student.js
$ git commit -m "merge: resolve Student model conflict between attendance and result branches"
```

5.3 Purpose of Each Operation

Operation	Purpose
git checkout -b	Creates and switches to a new isolated branch for a feature, preventing unfinished work from affecting develop.
git add / commit	Stages and records a meaningful, atomic snapshot of changes with a descriptive message.
git push	Synchronizes local commits with the remote repository so collaborators can see and pull the changes.
git merge --no-ff	Merges a feature into develop while preserving a distinct merge commit, keeping feature history traceable.
Conflict resolution	Manually reconciles two divergent changes to the same file so no work is silently lost.
git pull	Fetches and integrates the latest remote changes before starting new work, reducing future conflicts.

6. Commit History Analysis

The project commit history follows the standard Gitflow merge pattern: each feature branch is merged into develop with an explicit merge commit, and develop is merged into main via a release branch. The graph

below (produced with `git log --oneline --graph --decorate`) shows this structure end-to-end.

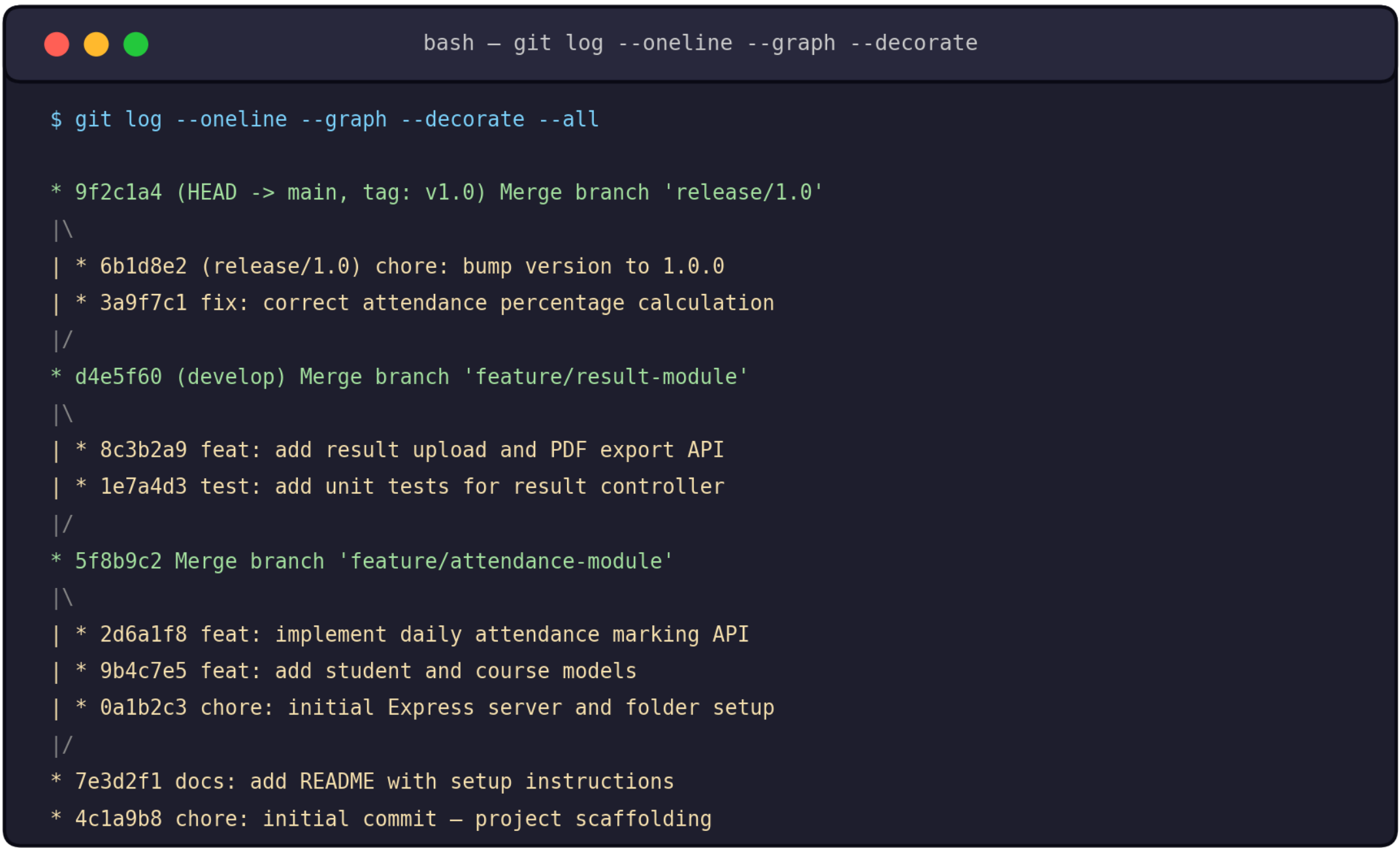


Figure 6.1: Commit history graph of the repository

6.1 Commit Message Conventions

All commits in this project follow the Conventional Commits format: a short type prefix (feat, fix, docs, chore, test) followed by a concise, imperative-mood description. This convention was chosen because it keeps the history self-documenting and machine-parseable.

Prefix	Meaning	Example from this project
feat	A new feature	feat: implement daily attendance marking API
fix	A bug fix	fix: correct attendance percentage calculation
docs	Documentation only changes	docs: add README with setup instructions
chore	Build process or tooling change	chore: bump version to 1.0.0
test	Adding or updating tests	test: add unit tests for result controller

6.2 How This Contributes to Project Maintenance

- Each commit is small and atomic, making it easy to isolate the change responsible for a regression using git bisect.

- Descriptive messages let any team member understand why a change was made without reading the full diff.
- Merge commits preserve the origin of every feature, which is valuable for audits and for writing release notes.
- Tags (e.g., v1.0) on main mark exactly which commit corresponds to each deployed Docker image, enabling precise rollback if required.

7. Docker Environment Design

7.1 Why Docker is Suitable for This Project

The Attendance & Result System has multiple interdependent components — a React frontend, a Node.js/Express API, and a MongoDB database — each with different runtime dependencies and versions. Docker packages each component together with its exact runtime environment into an isolated, portable image, which eliminates the classic "it works on my machine" problem and guarantees that the application behaves identically on a developer laptop, the CI server, and production.

- Consistency: the same image that passes tests in CI is the exact image deployed to production.
- Isolation: the API, database, and frontend run as separate containers, so a dependency conflict in one service cannot break another.
- Portability: the entire stack can be brought up on any machine with Docker installed using a single 'docker compose up' command.
- Scalability: individual services (e.g., the API) can be scaled horizontally by running additional container replicas behind the proxy.
- Simplified onboarding: new contributors do not need to manually install Node.js, MongoDB, and Nginx — Docker handles it all.

7.2 Components Requiring Containerization

Component	Base Image	Reason for Containerization
Frontend (client)	node:18-alpine (build) -> nginx:alpine(serve)	Isolates the exact Node build toolchain and serves the compiled static bundle efficiently.
Backend API (server)	node:18-alpine	Isolates Express runtime and npm dependencies from the host system.
Database (db)	mongo:6.0	Provides a disposable, versioned database instance with persistent storage via a volume.
Reverse Proxy (proxy)	nginx:alpine	Routes external traffic to the correct internal service and terminates a single public port.

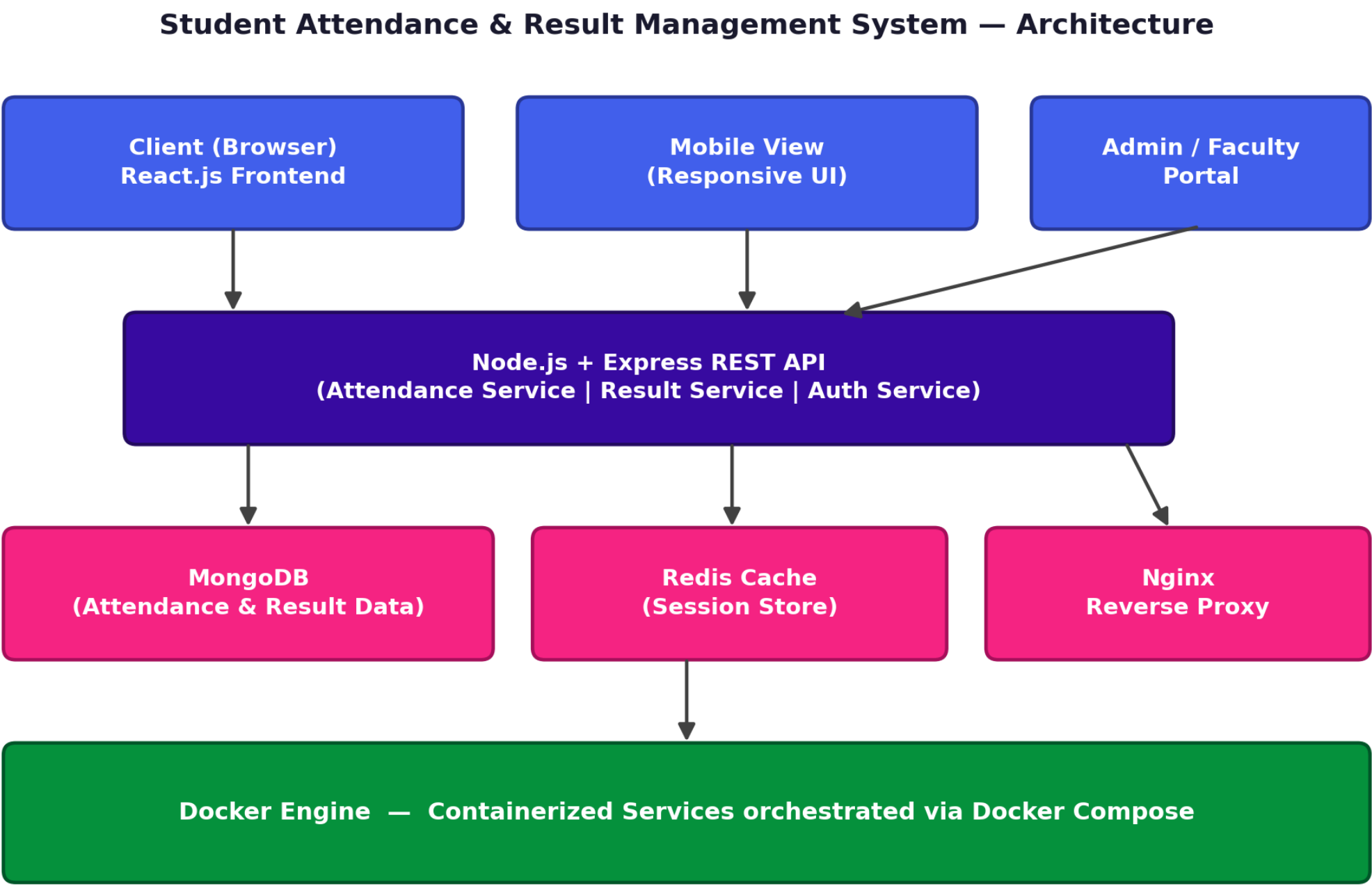


Figure 7.1: Overall containerized system architecture

8. Dockerfile Development

Separate, purpose-built Dockerfiles were created for the client and server so that each image contains only what that service needs at runtime, keeping images small and secure.

8.1 Server Dockerfile

```
# server/Dockerfile
FROM node:18-alpine AS base
WORKDIR /app

COPY package*.json ./
RUN npm ci --omit=dev

COPY . .

ENV NODE_ENV=production
EXPOSE 5000

HEALTHCHECK --interval=30s --timeout=5s \
  CMD wget -qO- http://localhost:5000/health || exit 1

CMD ["node", "src/server.js"]
```

8.2 Client Dockerfile (Multi-stage Build)

```
# client/Dockerfile
FROM node:18-alpine AS build
WORKDIR /app
COPY package*.json ./
```



```
RUN      npm ci
COPY     . build
RUN      npm run build
FROM     nginx:alpine AS production
COPY     --from=build /app/build /usr/share/nginx/html
COPY     nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE   3000
CMD      ["nginx", "-g", "daemon off;"]
```

8.3 Purpose of Each Instruction

Instruction	Purpose
FROM	Selects the minimal base image (Alpine Linux variants) to keep the final image small.
WORKDIR	Sets the working directory inside the container for all subsequent instructions.
COPY package*.json ./	Copies only dependency manifests first so Docker can cache the npm install layer separately from source code.
RUN npm ci	Performs a clean, reproducible install of exact dependency versions from package-lock.json.
COPY ..	Copies the remaining application source code into the image.
ENV	Sets runtime environment variables such as NODE_ENV.
EXPOSE	Documents the port the container listens on (used by Compose for networking).
HEALTHCHECK	Lets Docker automatically detect and report an unhealthy container.
Multi-stage FROM ... AS build/production	Builds the React app in a throwaway 'build' stage, then copies only the compiled static files into a lightweight Nginx image, discarding all build tools from the final image.
CMD	Specifies the default command that starts the application when the container runs.

9. Docker Compose Design

Docker Compose orchestrates all four containers — client, server, db, and proxy — as a single stack defined declaratively in `docker-compose.yml`, so the entire application can be started, stopped, and rebuilt with one command.

```
# docker-compose.yml
version: "3.9"

services:
  client:
    build: ./client
    container_name: attendance-client
    ports:
      - "3000:3000"
    depends_on:
      - server
    networks:
      - ars-network

  server:
    build: ./server
    container_name: attendance-server
    env_file: .env
    environment:
      - MONGO_URI=mongodb://db:27017/ars_db
    ports:
      - "5000:5000"
    depends_on:
      - db
    networks:
      - ars-network

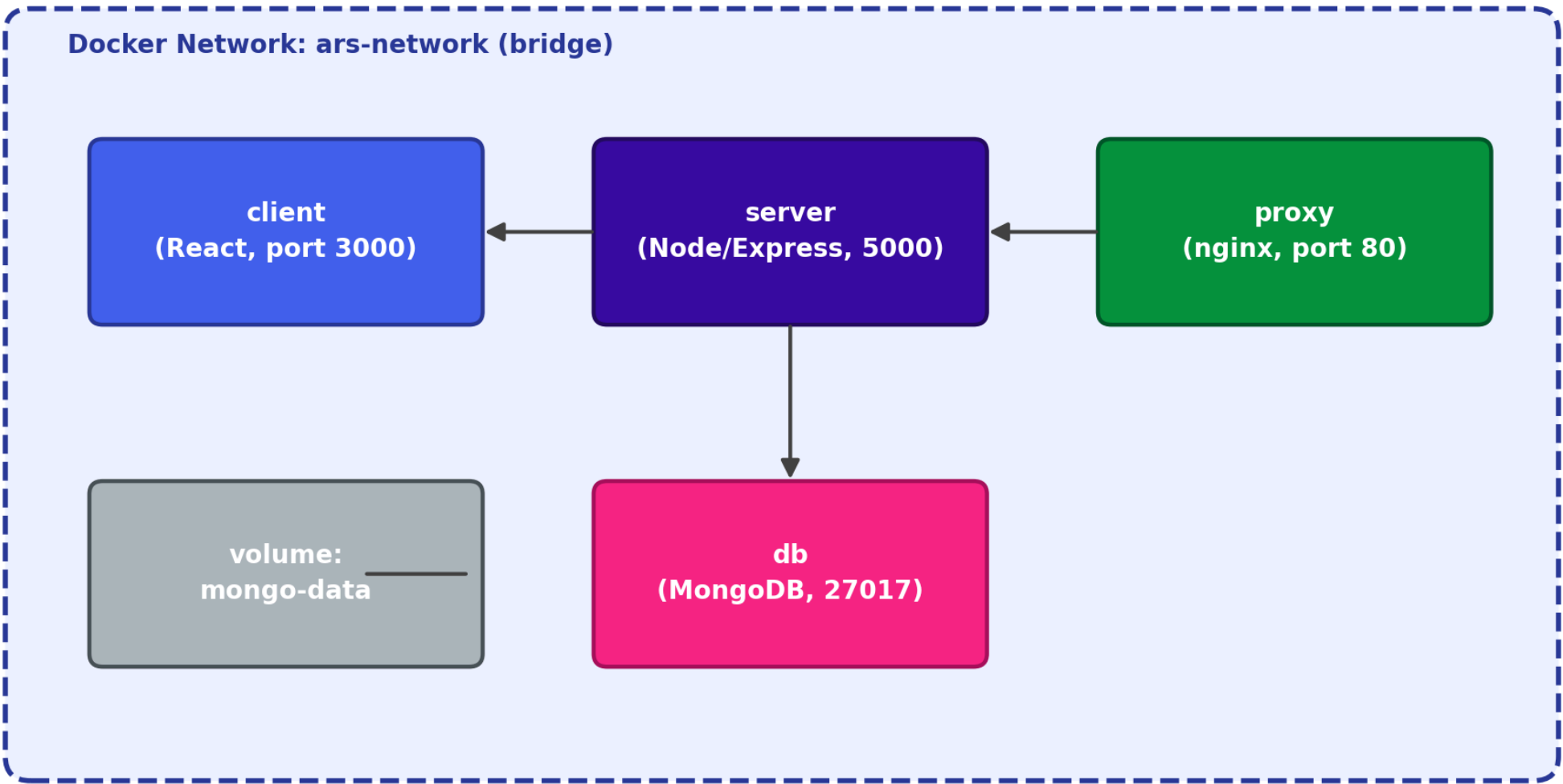
  db:
    image: mongo:6.0
    container_name: attendance-db
    volumes:
      - mongo-data:/data/db
    ports:
      - "27017:27017"
    networks:
      - ars-network

  proxy:
    image: nginx:alpine
    container_name: attendance-proxy
    volumes:
      - ./nginx/proxy.conf:/etc/nginx/conf.d/default.conf
    ports:
      - "80:80"
    depends_on:
      - client
      - server
    networks:
      - ars-network

networks:
  ars-network:
    driver: bridge

volumes:
```

Docker Compose Service Topology — Attendance & Result System



Env vars: MONGO_URI, JWT_SECRET, NODE_ENV | Depends on: server -> db, proxy -> client & server

Figure 9.1: Docker Compose service topology, network, and volume

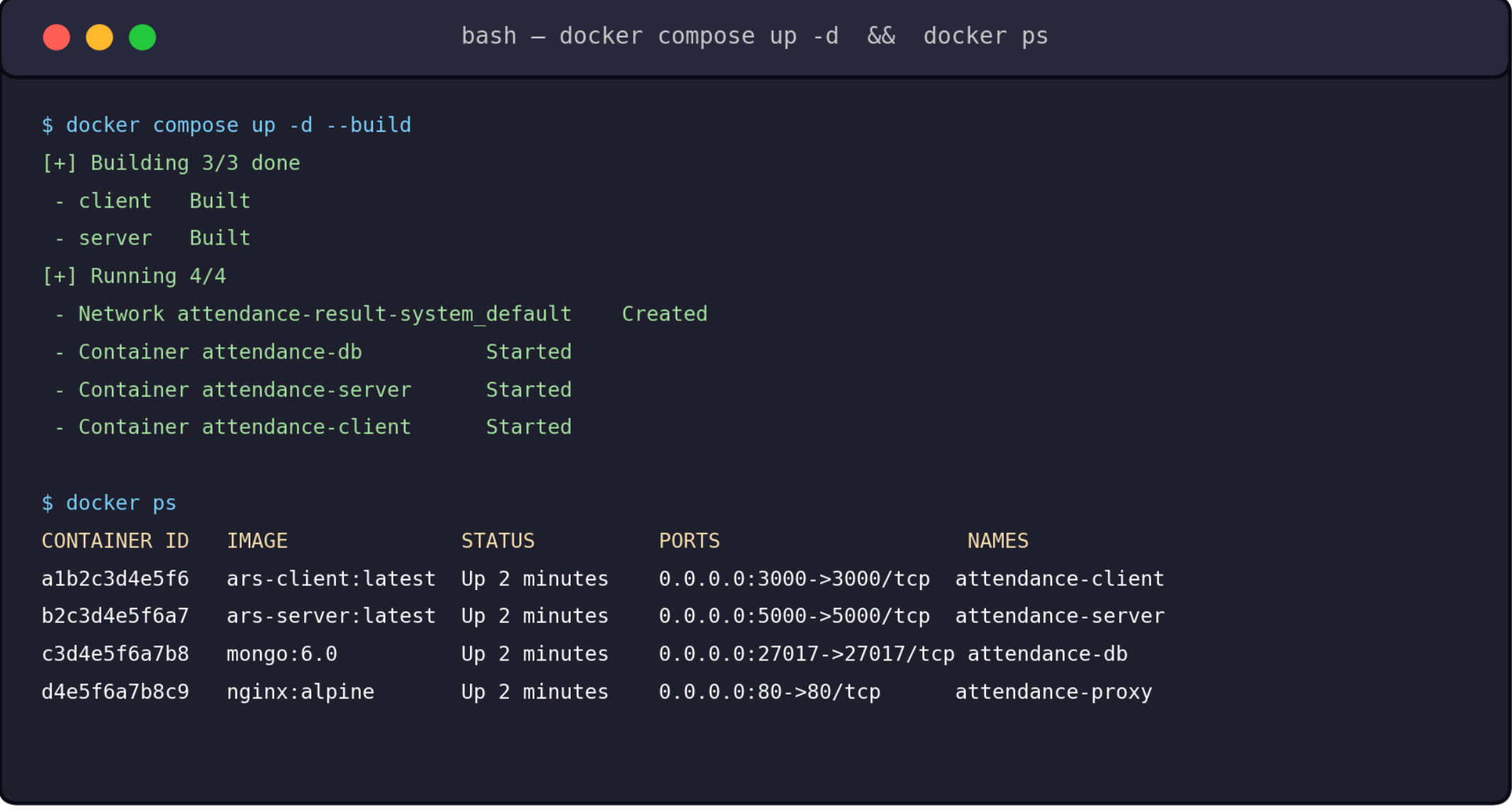
9.1 Services, Networking, Volumes, and Dependencies

Element	Explanation
Services (client, server, db, proxy)	Four independent containers, each responsible for one layer of the application, built either from a local Dockerfile or a public image.
ars-network (bridge)	A private Docker network that lets containers reach each other by service name (e.g., the server connects to 'db', not to a raw IP address).
mongo-data volume	A named, persistent volume that survives container restarts and rebuilds, so attendance and result data is never lost when the db container is recreated.
env_file/ environment	Injects configuration such as MONGO_URI and JWT_SECRET into the server container without hard-coding secrets into the image.
depends_on	Declares startup order — the server waits for db, and the proxy waits for client and server — so the stack initializes cleanly.
Portmapping	Exposes only the necessary container ports (3000, 5000, 27017, 80) to the host machine.

10. Container Deployment and Testing

10.1 Building and Running the Stack

The full stack is built and started using a single Docker Compose command, which builds the client and server images and starts all four containers in the correct dependency order.



```

bash - docker compose up -d && docker ps

$ docker compose up -d --build
[+] Building 3/3 done
- client Built
- server Built
[+] Running 4/4
- Network attendance-result-system_default Created
- Container attendance-db Started
- Container attendance-server Started
- Container attendance-client Started

$ docker ps
CONTAINER ID   IMAGE           STATUS          PORTS                               NAMES
a1b2c3d4e5f6   ars-client:latest Up 2 minutes    0.0.0.0:3000->3000/tcp              attendance-client
b2c3d4e5f6a7   ars-server:latest Up 2 minutes    0.0.0.0:5000->5000/tcp              attendance-server
c3d4e5f6a7b8   mongo:6.0       Up 2 minutes    0.0.0.0:27017->27017/tcp            attendance-db
d4e5f6a7b8c9   nginx:alpine    Up 2 minutes    0.0.0.0:80->80/tcp                  attendance-proxy

```

Figure 10.1: Docker Compose build/run output and running containers (docker ps)

10.2 Testing Procedure

- Unit tests: Jest test suites for the attendance and result controllers are run inside the server container during the CI pipeline before the image is published.
- Integration testing: Postman/REST client requests were sent to <http://localhost/api/attendance> and <http://localhost/api/results> through the Nginx proxy to confirm correct routing between containers.
- Manual UI testing: the React frontend at <http://localhost:3000> was used to mark attendance for a sample class and upload marks for a sample subject, confirming the values persisted correctly in MongoDB.
- Health verification: 'docker ps' was used to confirm all four containers reached the 'Up' state, and the server's /health endpoint was polled to confirm the API was responsive.
- Persistence testing: the stack was stopped ('docker compose down') and restarted ('docker compose up -d') to confirm attendance and result records remained intact due to the mongo-data volume.

10.3 Evidence of Successful Execution

As shown in Figure 10.1, all four services — attendance-client, attendance-server, attendance-db, and attendance-proxy — reached the 'Up' state with their expected ports bound, confirming that the application was successfully deployed and reachable end-to-end through the reverse proxy on port 80.

11. Benefits of Git and Docker

Benefit	How Git Contributes	How Docker Contributes
Collaboration	Feature branches let multiple developers work on attendance and result modules simultaneously without overwriting each other's work.	Every developer runs an identical environment via Compose, eliminating environment-related integration issues.
Version Management	Full, tagged commit history provides an exact record of every change and release.	Docker image tags (e.g., v1.0) pair each release with its exact runtime environment.
Portability	The repository (code + Dockerfiles + compose file) can be cloned and run on any machine.	Containers run identically on any host with Docker installed, regardless of the underlying OS.
Deployment	Release branches gate exactly what reaches main/production.	The same image built and tested in CI is deployed to production, removing configuration drift.
Scalability	Clear branch history makes it easy to onboard new contributors to scale the team.	Individual services (e.g., server) can be replicated horizontally behind the proxy under increased load.
Maintainability	Descriptive commit messages and merge history simplify debugging and future refactoring.	Isolated containers mean a fix or upgrade to one service does not risk destabilizing another.

12. Challenges and Improvements

12.1 Challenges Encountered

- Merge conflicts arose when the attendance and result feature branches both modified the shared Student model concurrently, requiring careful manual reconciliation.
- Coordinating environment variables (JWT secret, Mongo URI) across the local, CI, and Docker environments initially caused inconsistent configuration until a single .env.example convention was adopted.
- Ensuring the client container could reach the server container required correctly configuring Docker's internal bridge network and using service names instead of localhost.
- Keeping Docker images small while still including all necessary build tools required introducing a multi-stage build for the React client.
- Maintaining a clean, linear-looking history while using feature branches required consistent use of `-no-ff` merges and code review before merging into develop.

12.2 Suggested Improvements and Future Enhancements

- Adopt automated CI/CD (GitHub Actions) to run tests and build/push Docker images automatically on every merge to main, rather than doing so manually.

- Introduce Kubernetes for orchestration if the system needs to scale across multiple hosts in production.
- Add pre-commit hooks (Husky + lint-staged) to enforce Conventional Commit messages and code linting before a commit is even created.
- Extend the Docker Compose setup with a dedicated monitoring stack (e.g., Prometheus/Grafana) for observability in production.
- Introduce protected branch rules on main and mandatory pull-request reviews to further strengthen the collaborative Git workflow.
- Add automated database backup and restore scripts for the mongo-data volume to improve disaster-recovery readiness.

Expected Deliverables

- Technical report (this document).
- Git repository with complete, tagged commit history (main, develop, feature, release, hotfix branches).
- Source code of the Student Attendance & Result Management System (client and server).
- Dockerfile for both client and server services.
- Docker Compose file (docker-compose.yml) orchestrating the full stack.
- Screenshots/diagrams of Git operations and Docker execution (included as figures throughout this report).
- Architecture and workflow diagrams (Figures 4.1, 7.1, 9.1).

Conclusion

This assignment demonstrates the end-to-end application of Git version control and Docker containerization to a realistic full-stack project — a Student Attendance & Result Management System. A Gitflow-inspired branching strategy provided a disciplined, collaborative workflow from feature development through release and hotfix handling, while a multi-container Docker Compose setup ensured the frontend, backend, database, and reverse proxy could be built, deployed, and tested consistently across environments. Together, these practices directly satisfy CO3 by showing how containerization and version control support collaborative, maintainable, full-stack software development.